FRONTIER DENTAL · AI TEAM TAKE-HOME

Agentic Product Scraping & Structured Catalog Extraction

A runnable POC for Safco Dental — and a workflow designed to generalize to (almost) any site.

Deterministic where possible | AI where it pays | Honest & compliant | Production-minded

Two target categories · Agent-based design · Complete catalog via the site's own API · 30 passing tests

EXECUTIVE SUMMARY

01 What we built, at a glance

A working, agent-based scraper that discovers categories, extracts products, normalizes them to a documented schema, stores them (SQLite + JSON/CSV/Excel), and is architected to harden for production — with AI used only where it earns its place.

Two runtimes, one toolset

A deterministic core that runs with no API key, plus a conversational agent layer (terminal chat + Gradio web UI).

Self-adapting

One generic extractor driven by cached JSON "profiles". An LLM agent authors profiles for unseen sites — no per-site code.

Grounded & honest

Agents speak only from fetched pages / DB. Anti-bot blocks are detected and **not** evaded; a human-in-the-loop is the final tier.

Complete target data

Gloves **100** + Sutures **56** = **156** products with variant SKUs — the full displayed catalog, via the site's own API.

Proven generality

Live any-site + pagination demo on a permission-friendly sandbox (60 products across 3 pages, no JSON-LD).

Engineered

Rate-limit, retries, resumable checkpoints, dedup, idempotency, config-driven, secrets, observability, Docker — and 30 tests.

HOW WE GOT HERE — THINKING & ITERATION

02 The journey

STEP	WHAT WE FOUND / DID
Deterministic core	Safco serves clean schema.org **JSON-LD** in static HTML → extract 30 products at ~0.89 coverage with **zero LLM**. AI reserved for the hard parts.
Made it agentic	Added a conversational **conductor** (front door), **any-site** auto-authoring of extraction profiles, and a **Gradio** web UI.
Pagination puzzle	Category pages show only ~15 items; ?page= is client-side JS. Built **param-agnostic** next-page detection; proved pagination live on a sandbox (60 / 3 pages).
Compliance test	Pointed the tool at **Frontier's own site** → Cloudflare **403** + robots disallows AI crawlers. The tool **detects & refuses to evade**, and hands off to a human.
Completeness	Realised 15 ≠ the full catalog. Discovered the site's own **Algolia API** (observe-and-replay) → true totals **Gloves 100 / Sutures 56** (gloves matches the browser's "7 pages, 100 products").
New agent	This taught us the missing piece: a **completeness-critic** agent that checks "did we get everything?" and escalates if not.

SOUND, AGENT-BASED DESIGN

03 Architecture at a glance

Seeds → Navigator → Extractor (profile) → Validator / Dedup → SQLite → Export JSON/CSV/XLSX

A conversational **conductor** sits on top, turning natural language into tool calls; a **completeness-critic** checks the result; a **reporter** answers questions over the data — all grounded in real output.

AGENT	RESPONSIBILITY
Navigator	Discover products, subcategories, pagination (param-agnostic).
Page-classifier	Category / listing / detail (rules first, LLM if ambiguous).
Profile-author ★	Inspect an unseen template, write + self-validate + cache an extraction profile.
Extractor	Apply the cached profile; LLM fallback with a grounding guard.
Validator / dedup	Normalize to schema, dedup on SKU, score field coverage.
Completeness-critic ★	Compare what we captured vs the page's true total; escalate if incomplete.
Conductor ★	Conversational entry point over any site (chat / web UI).
Reporter	Grounded natural-language Q&A over the catalog.

★ = where AI/agents add the most value

THE CORE IDEA

04 Completeness + autonomous discovery

A naïve scraper grabs the 15-item sample and thinks it's done. The agentic system should **know it isn't**, and find the rest.

1 · Completeness-critic

Compares extracted count to the page's **true total**. On Safco it autonomously reports **"15 of 100 — incomplete"** with no human hint.

2 · Observe & replay

A real browser captures the page's **own data API call** (Algolia). The captured request already contains the right filters & `nbHits=100` — replay it paginated to get everything.

3 · Cache the recipe

Store the discovered API recipe like any profile — learned once, deterministic after. Generalizes to any API-driven catalog.

Honest framing

For this 24-hour POC we **hand-authored the Safco/Algolia recipe** together with the AI agents — a fast path for the known target. That is not the whole system; it is an example of what the workflow produces. The agents *discover and cache* such recipes, and as they meet more sites, coverage approaches "any site that permits automation." The browser/MCP "vision" tier is the general (slower, costlier) solver we propose for the hardest cases.

THE RIGHT KIND OF ENGINEERING JUDGMENT

05 Honesty & compliance

We tested the tool against **Frontier Dental's own website**. The result is a feature, not a failure.

- The site is **Cloudflare bot-protected** — a classic Python scraper (and ours) gets **HTTP 403** even with full browser headers.
- Its `robots.txt` **disallows AI crawlers** (ClaudeBot, GPTBot, CCBot…) and signals `ai-train=no`.
- So the tool **detects the block, refuses to evade it**, and escalates to a human — rather than circumventing access controls.

> "blocked: HTTP 403 (anti-bot / access denied) — refusing to evade. A real browser tier + site permission would be required."
> — the tool's own dead-letter / human-handoff record

Demonstrating that the system **respects access controls and asks for help** is exactly the production judgment the role calls for — far stronger than bypassing a potential employer's protections.

EVIDENCE

06 Findings & output

TARGET	STATIC SAMPLE	TRUE CATALOG	HOW
Dental Exam Gloves	15 (curated)	**100 products**	Site's Algolia API · matches the browser's 7 pages × ~15
Sutures & surgical	15 (curated)	**56 products**	Site's Algolia API · with variant SKUs captured
books.toscrape.com	—	**60 / 3 pages**	Live any-site demo · CSS profile · real pagination · no JSON-LD
frontierdental.com	—	**0 — blocked**	Cloudflare 403 + AI-disallowed robots → compliance handoff

156
complete target products (with variants), normalized & exported

30
automated tests passing (offline fixtures)

JSON · CSV · XLSX · SQLite
documented schema + per-run data-quality report

Sample datasets committed in the repo

BUILT TO HARDEN

07 Production-minded design

Every item the brief asked for, mapped to a real mechanism:

✓ Rate limiting + jitter · ✓ Retries (backoff) · ✓ Error isolation · ✓ Resumable checkpointing · ✓ Dead-letter queue · ✓ Dedup on SKU · ✓ Idempotent upsert · ✓ Config-driven · ✓ Secrets via .env · ✓ Structured JSON logs · ✓ Coverage / drift metrics · ✓ robots.txt aware · ✓ Anti-bot compliance · ✓ Human-in-the-loop · ✓ Dockerfile · ✓ Deployment path

Extraction escalation ladder

Tier 1 · JSON-LD (deterministic, cheap)

Tier 2 · CSS rules (profile-driven)

Tier 3 · LLM extraction fallback (grounded)

Tier 4 · Observe-and-replay the site's data API

Tier 5 · Browser / MCP “vision” tier (slower, costlier)

Final · Human handoff — when blocked or uncertain, ask, don't guess or evade

FROM POC TO PRODUCT

08 Vision & roadmap

Robustness

Sitemap discovery; automatic browser/vision-tier escalation; distributed frontier + workers; profile versioning; scheduled re-crawls.

Self-improving coverage

The completeness-critic guarantees we never silently ship partial data. Each new site teaches the recipe cache → coverage trends toward "any site that permits it."

Real-business integration

Scheduled collection → data warehouse / BI; price-drop & new-SKU & out-of-stock alerts to Slack / email; the grounded reporter as a natural-language analytics layer.

System integrations

A small REST / MCP API so procurement, ERP, CRM and pricing engines can query the catalog or trigger a crawl; webhooks on catalog changes.

Coordination at scale

Today: a deterministic orchestrator for the known pipeline + an LLM conductor for open-ended calls + a completeness-critic for QA. Next: grow the conductor into a **budget-aware supervisor** (orchestrator-worker) that decomposes goals, dispatches workers, and re-plans on failure — an LLM manager only where decisions are *uncertain*, never just because there are many agents.

The thesis: deterministic speed for the easy 90%, AI for the hard 10%, a completeness-critic to know the difference, and a human as the honest final tier.

IN ONE LINE

A working prototype that is honest about what it does, and clear about how it becomes everything it could.

Sound agentic architecture | AI where it adds value | Compliant by design | A real path to production

Deliverables: Git repo · README + User Manual · architecture & schema docs · sample datasets · this deck.